

Resolução Comentada

Apêndice D

Programação de Jogos:
Solução de Sudoku

Baseado na obra

C: Como Programar

Paul Deitel • Harvey Deitel

6ª edição — Pearson Prentice Hall, 2011

Sobre esta resolução. Este documento apresenta a resolução completa e didática dos oito desafios de programação propostos no Apêndice D da obra *C: Como Programar*. Cada desafio é desenvolvido no estilo incremental característico dos autores — enunciado, análise, algoritmo, implementação comentada em C, saída esperada e observações pedagógicas (boas práticas, erros comuns, dicas de desempenho). Como o Apêndice D sintetiza conceitos que vão de arrays bidimensionais (Cap. 6) até recursão e ponteiros (Caps. 5 e 7), o material aqui apresentado utiliza apenas recursos da linguagem C apresentados até esses capítulos.

Sumário

D Programação de Jogos: Solução de Sudoku	3
D.1 Visão geral e infraestrutura comum	3
D.1.1 Convenções de representação do tabuleiro	3
D.1.2 Função utilitária <code>ehMovimentoValido</code>	4
D.1.3 Função utilitária <code>imprimirTabuleiro</code>	5
D.2 Desafio 1: a função <code>validSudoku</code>	6
D.2.1 Análise do problema	6
D.2.2 Algoritmo	6
D.2.3 Implementação	7
D.2.4 Programa de teste e saída esperada	8
D.3 Desafio 2: solucionador por força bruta com retrocesso	10
D.3.1 Análise do problema	10
D.3.2 Algoritmo	10
D.3.3 Implementação	11
D.3.4 Saída esperada	12
D.3.5 Comparação com a versão “ingênua” de 81 laços aninhados	13
D.4 Desafio 3: gerador por permutações aleatórias	14
D.4.1 Análise do problema	14
D.4.2 Geração de uma permutação aleatória	14
D.4.3 Variante de <code>validSudoku</code> com verificação seletiva	15
D.4.4 Programa gerador completo	16
D.4.5 Discussão do experimento	17
D.4.6 Variante prática — usando retrocesso para construir um Sudoku válido	17
D.5 Desafio 4: solucionador “fácil” — estratégia dos singletons	18
D.5.1 Análise do problema	18
D.5.2 Algoritmo	18
D.5.3 Implementação	19
D.5.4 Discussão e limitações	22
D.5.5 Comentário sobre duplas, triplas e suas versões ocultas	22
D.6 Desafio 5: heurística “esperar antes de tomar uma decisão”	23
D.6.1 Análise do problema	23
D.6.2 Algoritmo	23

D.6.3	Implementação	24
D.6.4	Discussão de complexidade	25
D.6.5	Combinando heurística e retrocesso	26
D.7	Desafio 6: heurística da antecipação (look-ahead)	26
D.7.1	Análise do problema	26
D.7.2	Algoritmo	27
D.7.3	Implementação	27
D.7.4	Custo e benefício	29
D.8	Desafio 7: criando quebra-cabeças com células vazias	29
D.8.1	Análise do problema	29
D.8.2	Algoritmo (esvaziamento aleatório simples)	30
D.8.3	Algoritmo (esvaziamento simétrico)	30
D.8.4	Implementação	30
D.8.5	Sobre dificuldade e solvibilidade	31
D.9	Desafio 8: demonstrando que um Sudoku tem solução única	31
D.9.1	Análise do problema	31
D.9.2	Algoritmo	32
D.9.3	Implementação	32
D.9.4	Discussão da poda	33
D.9.5	Aplicação prática: refinando o gerador de quebra-cabeças	33
D.10	Programa integrado e demonstração	35
D.11	Conclusão	36

APÊNDICE D

Programação de Jogos: Solução de Sudoku

D.1 Visão geral e infraestrutura comum

O Apêndice D apresenta um conjunto de desafios de programação cujo objetivo é, ao mesmo tempo, divertir o leitor e exercitar todos os mecanismos que foram introduzidos nos capítulos anteriores: arrays bidimensionais, funções, ponteiros, estruturas de iteração aninhadas, geração de números pseudoaleatórios e — onde o problema exigir — recursão. Antes de atacarmos cada desafio individualmente, é prudente construir uma pequena infraestrutura comum a todos eles. Essa estratégia é, aliás, a mesma adotada em todo o livro: identificamos as funcionalidades reutilizáveis, programamo-las uma única vez e, a partir daí, compomos soluções cada vez mais sofisticadas.

[gear] Observação de Engenharia de Software

Reaproveitar funções utilitárias é a essência da boa engenharia de software. Em vez de duplicar a verificação “este movimento é válido?” em cada solucionador, escrevemos a função uma única vez e a invocamos onde for necessário. Quando uma falha aparecer (ou um aprimoramento for desejado), basta corrigir um único ponto do código.

D.1.1 Convenções de representação do tabuleiro

Seguindo a recomendação do Apêndice D, representamos o tabuleiro como um array bidimensional de inteiros de dimensões 10×10 , ignorando a *linha 0* e a *coluna 0*. Assim, a célula da i -ésima linha e j -ésima coluna corresponde a `s[i][j]` com $1 \leq i, j \leq 9$, em conformidade com a numeração tradicional da comunidade do Sudoku. Uma célula vazia é representada pelo valor `0`; uma célula preenchida contém um dos dígitos `1` a `9`.

[check] Boa Prática de Programação

Definir constantes simbólicas (*macros*) para o tamanho do tabuleiro elimina “números mágicos” espalhados pelo código. Caso desejássemos, no futuro, generalizar para um tabuleiro 16×16 , alteraríamos apenas as macros.

D.1.2 Função utilitária ehMovimentoValido

Praticamente todos os solucionadores precisarão responder à pergunta: “Se eu colocar o dígito d na célula $s[i][j]$, isso violará alguma regra do Sudoku?” Em vez de reverificar o tabuleiro inteiro a cada tentativa, é muito mais eficiente verificar apenas a linha i , a coluna j e a grade de 3×3 à qual $s[i][j]$ pertence. A Listagem D.1 apresenta o cabeçalho do programa, as constantes e essa função auxiliar.

```

1  /* sudoku_infra.h -- infraestrutura comum aos desafios do Apêndice D.
   */
2  #ifndef SUDOKU_INFRA_H
3  #define SUDOKU_INFRA_H
4
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <time.h>
8
9  #define LADO          9      /* dimensão do Sudoku padrão */
10 #define DIM           10     /* dimensão do array; ignoramos índice 0 */
11 #define VAZIO         0     /* valor que indica célula em branco */
12 #define BLOCO         3     /* dimensão das sub-grades 3x3 */
13
14 /* Verifica se colocar o dígito d em s[linha][col] mantém o tabuleiro
   */
15 /* consistente com as regras do Sudoku. Retorna 1 se sim, 0 se não.
   */
16 int ehMovimentoValido(int s[DIM][DIM], int linha, int col, int d)
17 {
18     int k, i, j;
19     int inicioL, inicioC;
20
21     /* 1) o dígito já apareceu na linha? */
22     for (k = 1; k <= LADO; k++)
23         if (s[linha][k] == d) return 0;
24
25     /* 2) o dígito já apareceu na coluna? */
26     for (k = 1; k <= LADO; k++)
27         if (s[k][col] == d) return 0;
28
29     /* 3) o dígito já apareceu na grade 3x3 que contém [linha][col]?
       */
30     inicioL = ((linha - 1) / BLOCO) * BLOCO + 1;
31     inicioC = ((col - 1) / BLOCO) * BLOCO + 1;
32     for (i = inicioL; i < inicioL + BLOCO; i++)
33         for (j = inicioC; j < inicioC + BLOCO; j++)
34             if (s[i][j] == d) return 0;
35
36     return 1;      /* nenhum conflito */

```

```

37 }
38 #endif

```

Listing D.1: Macros, cabeçalhos e a função `ehMovimentoValido`.

A aritmética para localizar a grade 3×3 merece atenção. A expressão `((linha - 1) / 3) * 3 + 1` faz exatamente o que precisamos: subtraímos 1 para que as linhas 1, 2, 3 caiam em 0, 1, 2; dividimos por 3 (divisão inteira) para obter o índice do bloco (0, 1 ou 2); multiplicamos por 3 e somamos 1 para voltar à numeração de 1 a 9, obtendo a primeira linha da grade (1, 4 ou 7, respectivamente). O mesmo raciocínio se aplica à coluna.

[warning] Erro Comum de Programação

Um erro frequente é escrever `(linha / 3) * 3 + 1` sem o ajuste `- 1`. Para `linha == 3`, isso produziria 4, quando o início correto do bloco é 1. Sempre teste a aritmética de bloco com os valores de fronteira 1, 3, 4, 6, 7 e 9.

D.1.3 Função utilitária `imprimirTabuleiro`

Como o padrão da linguagem C não inclui recursos gráficos, optamos por uma representação textual cuidadosa, com separadores que destacam as sub-grades de 3×3 . A Listagem D.2 apresenta a implementação.

```

1  /* Imprime o tabuleiro com separadores entre as sub-grades 3x3. */
2  void imprimirTabuleiro(int s[DIM][DIM])
3  {
4      int i, j;
5
6      printf("\n    1 2 3 | 4 5 6 | 7 8 9\n");
7      printf("  +-----+-----+-----+\n");
8      for (i = 1; i <= LADO; i++) {
9          printf("%d | ", i);
10         for (j = 1; j <= LADO; j++) {
11             if (s[i][j] == VAZIO)
12                 printf(". ");
13             else
14                 printf("%d ", s[i][j]);
15             if (j % 3 == 0) printf("| ");
16         }
17         printf("\n");
18         if (i % 3 == 0)
19             printf("  +-----+-----+-----+\n");
20     }
21 }

```

Listing D.2: Impressão formatada do tabuleiro.

Com essa pequena infraestrutura — macros de domínio, função de validação incremental e função

de impressão — estamos prontos para enfrentar cada um dos desafios. Nas seções a seguir, supomos que essas funções já estão disponíveis (em um cabeçalho `sudoku_infra.h` incluído em cada programa).

D.2 Desafio 1: a função `validSudoku`

Enunciado do desafio

Crie a função `int validSudoku( int sudokuBoard[10][10] )` que recebe um tabuleiro de Sudoku como um array bidimensional de inteiros (ignorando a linha 0 e a coluna 0). A função deve retornar 1 se um tabuleiro *completo* for válido, 2 se um tabuleiro *parcialmente completo* for válido e 0 se o tabuleiro for *inválido*.

D.2.1 Análise do problema

Antes de escrever uma única linha de código, é essencial entender com precisão o que “válido” significa neste contexto. Um Sudoku é válido quando, ao mesmo tempo:

1. **Todas as células contêm valores em $\{0, 1, 2, \dots, 9\}$.** O valor 0 representa célula vazia; qualquer outro valor fora desse intervalo é, por si só, um erro.
2. **Em cada linha, cada dígito de 1 a 9 aparece, no máximo, uma vez.** Em um tabuleiro completo, todos os nove dígitos devem aparecer exatamente uma vez; em um tabuleiro parcial, basta que não haja repetição entre as células já preenchidas.
3. **O mesmo vale para cada coluna.**
4. **O mesmo vale para cada uma das nove grades de 3×3 .**

A distinção entre “completo e válido” (retorno 1), “parcial e válido” (retorno 2) e “inválido” (retorno 0) sugere naturalmente o seguinte plano: percorremos todas as células para detectar duplicatas; ao mesmo tempo, registramos se existe pelo menos uma célula vazia. Ao final, se algum conflito foi detectado, retornamos 0; se nenhum conflito foi detectado e existem células vazias, retornamos 2; se nenhum conflito foi detectado e todas as 81 células estão preenchidas, retornamos 1.

D.2.2 Algoritmo

1. **Validar o domínio dos valores.** Percorra todas as células. Se alguma célula contiver valor fora de $\{0, 1, \dots, 9\}$, retorne 0. Caso contrário, marque `temVazio = 1` se encontrar pelo menos uma célula com valor 0.
2. **Verificar repetições nas nove linhas.** Para cada linha, utilize um array auxiliar `presente[10]` (índices 1 a 9) para marcar quais dígitos já apareceram. Se um dígito já estiver marcado, retorne 0.
3. **Verificar repetições nas nove colunas.** Procedimento análogo ao das linhas, trocando os papéis de linha e coluna.

4. **Verificar repetições nas nove sub-grades 3×3 .** Use dois laços externos (sobre os blocos 0, 1, 2) e dois internos (sobre as três linhas e três colunas dentro do bloco).
5. **Determinar o veredito.** Se chegamos até aqui, não há conflito. Retorne 2 se `temVazio == 1`, ou 1 se o tabuleiro estiver completo.

[check] Boa Prática de Programação

Reinicializar o vetor `presente` a cada linha (ou coluna, ou bloco) é o passo mais facilmente esquecido nessa rotina. Se o array não for zerado, um dígito que apareceu na linha 1 pode ser interpretado como repetição na linha 2, gerando falsos positivos.

D.2.3 Implementação

```

1  /* validSudoku.c -- desafio 1 do Apêndice D. */
2  /* Retorna: 1 se completo e válido, 2 se parcial e válido, */
3  /*          0 se houver qualquer violação das regras. */
4  #include "sudoku_infra.h"
5
6  int validSudoku(int sudokuBoard[DIM][DIM])
7  {
8      int i, j, k, valor;
9      int presente[DIM]; /* presente[d] != 0 se d já foi visto */
10     int temVazio = 0;
11     int blocoLinha, blocoCol;
12
13     /* 1) Validação de domínio dos valores. */
14     for (i = 1; i <= LADO; i++) {
15         for (j = 1; j <= LADO; j++) {
16             valor = sudokuBoard[i][j];
17             if (valor < 0 || valor > LADO)
18                 return 0; /* valor inválido */
19             if (valor == VAZIO)
20                 temVazio = 1;
21         }
22     }
23
24     /* 2) Verificação das nove linhas. */
25     for (i = 1; i <= LADO; i++) {
26         for (k = 0; k < DIM; k++) presente[k] = 0;
27         for (j = 1; j <= LADO; j++) {
28             valor = sudokuBoard[i][j];
29             if (valor != VAZIO) {
30                 if (presente[valor]) /* já apareceu */
31                     return 0;
32                 presente[valor] = 1;
33             }
34         }
35     }

```

```

36
37  /* 3) Verificação das nove colunas. */
38  for (j = 1; j <= LADO; j++) {
39      for (k = 0; k < DIM; k++) presente[k] = 0;
40      for (i = 1; i <= LADO; i++) {
41          valor = sudokuBoard[i][j];
42          if (valor != VAZIO) {
43              if (presente[valor])
44                  return 0;
45              presente[valor] = 1;
46          }
47      }
48  }
49
50  /* 4) Verificação das nove sub-grades 3x3. */
51  for (blocoLinha = 0; blocoLinha < BLOCO; blocoLinha++) {
52      for (blocoCol = 0; blocoCol < BLOCO; blocoCol++) {
53          for (k = 0; k < DIM; k++) presente[k] = 0;
54          for (i = 1; i <= BLOCO; i++) {
55              for (j = 1; j <= BLOCO; j++) {
56                  valor = sudokuBoard[blocoLinha*BLOCO + i]
57                          [blocoCol *BLOCO + j];
58                  if (valor != VAZIO) {
59                      if (presente[valor])
60                          return 0;
61                      presente[valor] = 1;
62                  }
63              }
64          }
65      }
66  }
67
68  /* 5) Veredito final. */
69  return temVazio ? 2 : 1;
70 }

```

Listing D.3: Implementação completa da função validSudoku.

D.2.4 Programa de teste e saída esperada

A Listagem D.4 apresenta um pequeno programa que constrói três tabuleiros — um inválido, um parcial e um completo — e invoca `validSudoku` em cada um deles.

```

1  int main(void)
2  {
3      /* Tabuleiro completo e VÁLIDO. */
4      int completo[DIM][DIM] = {
5          { 0 },

```

```

6      { 0, 5,3,4, 6,7,8, 9,1,2 },
7      { 0, 6,7,2, 1,9,5, 3,4,8 },
8      { 0, 1,9,8, 3,4,2, 5,6,7 },
9      { 0, 8,5,9, 7,6,1, 4,2,3 },
10     { 0, 4,2,6, 8,5,3, 7,9,1 },
11     { 0, 7,1,3, 9,2,4, 8,5,6 },
12     { 0, 9,6,1, 5,3,7, 2,8,4 },
13     { 0, 2,8,7, 4,1,9, 6,3,5 },
14     { 0, 3,4,5, 2,8,6, 1,7,9 }
15 };
16 /* Tabuleiro parcial e válido (várias células vazias). */
17 int parcial[DIM][DIM] = {
18     { 0 },
19     { 0, 5,3,0, 0,7,0, 0,0,0 },
20     { 0, 6,0,0, 1,9,5, 0,0,0 },
21     { 0, 0,9,8, 0,0,0, 0,6,0 },
22     { 0, 8,0,0, 0,6,0, 0,0,3 },
23     { 0, 4,0,0, 8,0,3, 0,0,1 },
24     { 0, 7,0,0, 0,2,0, 0,0,6 },
25     { 0, 0,6,0, 0,0,0, 2,8,0 },
26     { 0, 0,0,0, 4,1,9, 0,0,5 },
27     { 0, 0,0,0, 0,8,0, 0,7,9 }
28 };
29 /* Tabuleiro INVÁLIDO: dois 5 na primeira linha. */
30 int invalido[DIM][DIM] = {
31     { 0 },
32     { 0, 5,5,0, 0,7,0, 0,0,0 }, /* erro proposital */
33     { 0, 6,0,0, 1,9,5, 0,0,0 },
34     { 0, 0,9,8, 0,0,0, 0,6,0 },
35     { 0, 8,0,0, 0,6,0, 0,0,3 },
36     { 0, 4,0,0, 8,0,3, 0,0,1 },
37     { 0, 7,0,0, 0,2,0, 0,0,6 },
38     { 0, 0,6,0, 0,0,0, 2,8,0 },
39     { 0, 0,0,0, 4,1,9, 0,0,5 },
40     { 0, 0,0,0, 0,8,0, 0,7,9 }
41 };
42
43 printf("Tabuleiro completo -> validSudoku = %d\n",
44       validSudoku(completo));
45 printf("Tabuleiro parcial -> validSudoku = %d\n",
46       validSudoku(parcial));
47 printf("Tabuleiro invalido -> validSudoku = %d\n",
48       validSudoku(invalido));
49
50 return 0;
51 }

```

Listing D.4: Programa de teste para validSudoku.

Saída esperada:

```
1 Tabuleiro completo -> validSudoku = 1
2 Tabuleiro parcial -> validSudoku = 2
3 Tabuleiro invalido -> validSudoku = 0
```

[bolt] Dica de Desempenho

A complexidade dessa função é $\Theta(n^2)$, com $n = 9$ — ou seja, um custo constante na prática. Como ela será invocada milhões de vezes pelos solucionadores por força bruta, ainda assim vale a pena evitar trabalho desnecessário. Em situações em que apenas *uma* célula foi alterada, a função `ehMovimentoValido` (apresentada na Seção D.1) faz a mesma verificação examinando apenas 27 células, e por isso é claramente preferível dentro de laços de busca.

D.3 Desafio 2: solucionador por força bruta com retrocesso

Enunciado do desafio

A versão mais inteligente da técnica de força bruta consiste em verificar cada dígito antes de colocá-lo: se o tabuleiro permanece válido, prossiga para a próxima célula; se não, experimente outro dígito; se nenhum dos nove dígitos funcionar, retorne à célula anterior e troque seu valor. Implemente um solucionador com essa estratégia.

D.3.1 Análise do problema

A descrição acima é a definição clássica do algoritmo de *retrocesso* (em inglês, *backtracking*). A ideia é simples e poderosa: tentamos construir uma solução um passo de cada vez; sempre que percebemos que o caminho atual é inviável, “voltamos atrás” e tentamos outro. Como cada tentativa é validada antes de prosseguirmos, podemos enormes porções do espaço de busca — 9^{81} possibilidades cegas tornam-se algumas dezenas de milhares de tentativas inteligentes em um Sudoku típico.

A recursão é, sem dúvida, a forma mais elegante de expressar esse algoritmo. Aprendemos no Capítulo 5 que uma função recursiva resolve um problema maior reduzindo-o a uma versão menor de si mesmo. Aqui o “problema menor” é o tabuleiro com *uma célula a mais* preenchida.

[gear] Observação de Engenharia de Software

O retrocesso é onipresente em problemas combinatórios: o Passeio do Cavalo, as Oito Rainhas, palavras cruzadas, geração de permutações — todos seguem o mesmo esqueleto: “tente, valide, recurse, desfaça”. Reconhecer esse padrão é o primeiro passo para resolver dezenas de problemas aparentemente distintos com o mesmo arcabouço mental.

D.3.2 Algoritmo

1. **Encontre a primeira célula vazia.** Percorra o tabuleiro nas direções natural (linhas crescentes e, dentro de cada linha, colunas crescentes).

2. **Caso-base.** Se não houver célula vazia, o tabuleiro está completo e válido (pois cada dígito foi validado ao ser colocado). Retorne sucesso.
3. **Passo recursivo.** Para cada dígito d de 1 a 9:
 - (a) Use `ehMovimentoValido` para verificar se d pode ser colocado na célula corrente.
 - (b) Em caso afirmativo, atribua `s[i][j] = d` e chame recursivamente o solucionador.
 - (c) Se a chamada recursiva retornar sucesso, propague sucesso.
 - (d) Caso contrário, **desfaça** a atribuição (`s[i][j] = VAZIO`) e tente o próximo dígito.
4. Se nenhum dígito funcionou, retorne falha (isso obriga a chamada anterior a tentar outro dígito).

[warning] Erro Comum de Programação

Esquecer de *desfazer* a atribuição (`s[i][j] = VAZIO`) antes de tentar o próximo dígito é o erro mais comum em algoritmos de retrocesso. Sem essa restauração, o tabuleiro acumula valores “fantasmas” das tentativas anteriores, e o algoritmo simplesmente falha em todos os caminhos subsequentes.

D.3.3 Implementação

```

1  /* sudoku_backtracking.c -- desafio 2 do Apêndice D.                */
2  /* Resolve o tabuleiro modificando-o in place. Retorna 1 se uma    */
3  /* solução foi encontrada, 0 caso contrário.                      */
4  #include "sudoku_infra.h"
5
6  int resolverBacktracking(int s[DIM][DIM])
7  {
8      int i, j, d;
9
10     /* 1) Localiza a primeira célula vazia em ordem linha-coluna. */
11     for (i = 1; i <= LADO; i++) {
12         for (j = 1; j <= LADO; j++) {
13             if (s[i][j] == VAZIO) {
14
15                 /* 2) Tenta cada dígito de 1 a 9. */
16                 for (d = 1; d <= LADO; d++) {
17                     if (ehMovimentoValido(s, i, j, d)) {
18                         s[i][j] = d;                /* tente */
19                         if (resolverBacktracking(s)) /* recurse */
20                             return 1;
21                         s[i][j] = VAZIO;            /* desfaza */
22                     }
23                 }
24                 /* Nenhum dígito serviu: avisa o nível anterior. */
25                 return 0;
26             }
27         }
28     }
29 }
```

```

28     }
29     /* Nenhuma célula vazia: tabuleiro completo (caso-base). */
30     return 1;
31 }
32
33 int main(void)
34 {
35     /* Quebra-cabeça clássico publicado pelo The Times (Londres). */
36     int s[DIM][DIM] = {
37         { 0 },
38         { 0, 5,3,0, 0,7,0, 0,0,0 },
39         { 0, 6,0,0, 1,9,5, 0,0,0 },
40         { 0, 0,9,8, 0,0,0, 0,6,0 },
41         { 0, 8,0,0, 0,6,0, 0,0,3 },
42         { 0, 4,0,0, 8,0,3, 0,0,1 },
43         { 0, 7,0,0, 0,2,0, 0,0,6 },
44         { 0, 0,6,0, 0,0,0, 2,8,0 },
45         { 0, 0,0,0, 4,1,9, 0,0,5 },
46         { 0, 0,0,0, 0,8,0, 0,7,9 }
47     };
48
49     printf("Tabuleiro inicial:");
50     imprimirTabuleiro(s);
51
52     if (resolverBacktracking(s)) {
53         printf("\nSolucao encontrada:");
54         imprimirTabuleiro(s);
55     } else {
56         printf("\nNao ha solucao para esse tabuleiro.\n");
57     }
58     return 0;
59 }

```

Listing D.5: Solucionador por força bruta com retrocesso.

D.3.4 Saída esperada

```

1  Tabuleiro inicial:
2      1 2 3 | 4 5 6 | 7 8 9
3  +-----+-----+-----+
4  1 | 5 3 . | . 7 . | . . . |
5  2 | 6 . . | 1 9 5 | . . . |
6  3 | . 9 8 | . . . | . 6 . |
7  +-----+-----+-----+
8  4 | 8 . . | . 6 . | . . 3 |
9  5 | 4 . . | 8 . 3 | . . 1 |
10 6 | 7 . . | . 2 . | . . 6 |
11 +-----+-----+-----+
12 7 | . 6 . | . . . | 2 8 . |

```

```

13 8 | . . . | 4 1 9 | . . 5 |
14 9 | . . . | . 8 . | . 7 9 |
15 +-----+-----+-----+
16
17 Solucao encontrada:
18   1 2 3 | 4 5 6 | 7 8 9
19 +-----+-----+-----+
20 1 | 5 3 4 | 6 7 8 | 9 1 2 |
21 2 | 6 7 2 | 1 9 5 | 3 4 8 |
22 3 | 1 9 8 | 3 4 2 | 5 6 7 |
23 +-----+-----+-----+
24 4 | 8 5 9 | 7 6 1 | 4 2 3 |
25 5 | 4 2 6 | 8 5 3 | 7 9 1 |
26 6 | 7 1 3 | 9 2 4 | 8 5 6 |
27 +-----+-----+-----+
28 7 | 9 6 1 | 5 3 7 | 2 8 4 |
29 8 | 2 8 7 | 4 1 9 | 6 3 5 |
30 9 | 3 4 5 | 2 8 6 | 1 7 9 |
31 +-----+-----+-----+

```

[bolt] Dica de Desempenho

Este solucionador resolve, em frações de segundo, virtualmente qualquer Sudoku publicado em jornais. Sudokus considerados “mais difíceis do mundo” podem exigir centenas de milhares de chamadas recursivas, mas mesmo nesses casos o tempo total é tipicamente inferior a um segundo em um computador moderno. A simplicidade desse algoritmo — uma única função de cerca de 20 linhas — contrasta com as estratégias heurísticas elaboradas, e ilustra um princípio importante: *quando o hardware é farto, a simplicidade vence.*

D.3.5 Comparação com a versão “ingênua” de 81 laços aninhados

O Apêndice D menciona uma técnica alternativa: aninhar 81 laços `for`, cada um varrendo de 1 a 9. O número total de combinações é 9^{81} , ou aproximadamente $1,97 \times 10^{77}$. Para se ter ideia da escala, mesmo a 10^{12} tentativas por segundo — desempenho de um supercomputador — testar todas elas levaria cerca de 6×10^{57} anos, muitas ordens de magnitude superior à idade estimada do universo.

A versão com retrocesso “poda” esse espaço de busca explorando apenas caminhos viáveis. Em termos práticos, o número de combinações efetivamente visitadas cai para a faixa de algumas dezenas de milhares. Esse é o poder de uma decisão arquitetural correta.

D.4 Desafio 3: gerador por permutações aleatórias

Enunciado do desafio

Escreva a função `void permutations(int sudokuBoard[10][10])` que recebe um array bidimensional de 10×10 e preenche cada uma das nove linhas da grade 9×9 com uma permutação selecionada aleatoriamente dos dígitos de 1 a 9. Em seguida, use `validSudoku` em laço até obter um Sudoku válido. Inclua uma opção em `validSudoku` que a faça verificar apenas colunas e sub-grades de 3×3 .

D.4.1 Análise do problema

Esta abordagem tem um forte sabor de *Monte Carlo*: tentamos configurações aleatórias até que uma delas, por sorte, seja válida. A probabilidade de uma única tentativa produzir um Sudoku válido é extremamente baixa, mas *não nula*; com tempo suficiente, o gerador eventualmente terá sucesso.

A ideia, descrita no apêndice, é gerar uma permutação dos dígitos de 1 a 9 para cada linha. Como cada linha é, por construção, uma permutação, não há repetições *dentro de uma mesma linha*. A função `validSudoku` só precisa, portanto, verificar duas coisas: que não há repetições nas colunas e que não há repetições nas sub-grades 3×3 . Daí a opção adicional que vamos introduzir.

[bolt] Dica de Desempenho

A probabilidade de uma combinação aleatória de nove permutações constituir um Sudoku válido é da ordem de 10^{-21} . Ou seja, na prática, este gerador é didático — ilustra perfeitamente o que “força bruta” significa — mas é absolutamente impraticável como gerador real. Em programas sérios, combina-se este método com retrocesso (vide Seção D.8).

D.4.2 Geração de uma permutação aleatória

O Apêndice D descreve o método ingênuo: “escolha um dígito; para o próximo, escolha repetidamente até que seja diferente do anterior; e assim por diante”. Implementamos exatamente esse método na Listagem D.6.

```
1  /* Preenche linha[1..9] com uma permutação aleatória de 1..9.  */
2  void permutaLinha(int linha[DIM])
3  {
4      int usado[DIM];
5      int pos, d;
6
7      /* Marca todos os dígitos como ainda não utilizados. */
8      for (d = 0; d < DIM; d++) usado[d] = 0;
9
10     /* Para cada posição 1..9, sorteia repetidamente até obter */
11     /* um dígito ainda não utilizado.                          */
12     for (pos = 1; pos <= LADO; pos++) {
13         do {
14             d = (rand() % LADO) + 1;    /* sorteia em [1, 9] */
15         } while (usado[d]);
```

```

16         usado[d] = 1;
17         linha[pos] = d;
18     }
19 }
20
21 /* Aplica permutaLinha a cada uma das nove linhas do tabuleiro. */
22 void permutaLinha(int sudokuBoard[DIM][DIM])
23 {
24     int i;
25     for (i = 1; i <= LADO; i++)
26         permutaLinha(sudokuBoard[i]);
27 }

```

Listing D.6: Geração de uma linha como permutação aleatória.

[check] Boa Prática de Programação

O laço `do...while` aqui é justo: *precisamos* executar o sorteio pelo menos uma vez antes de testar a condição. Esse é o critério clássico para escolher entre `while` e `do...while` — se o teste deve ocorrer *depois* da primeira execução do corpo, use `do...while`.

D.4.3 Variante de validSudoku com verificação seletiva

O apêndice nos pede para introduzir uma opção que faça a função verificar *apenas* colunas e sub-grades. Há várias maneiras de implementar isso; optamos por uma flag inteira que recebe um padrão de bits (estilo clássico de C): bit 0 ativa a verificação de linhas, bit 1 a de colunas, bit 2 a de sub-grades. Para verificar todas as três, passamos `0x7`; para verificar apenas colunas e sub-grades, passamos `0x6`.

```

1  #define VERIFICA_LINHAS  0x1
2  #define VERIFICA_COLUNAS 0x2
3  #define VERIFICA_BLOCOS  0x4
4  #define VERIFICA_TUDO
   (VERIFICA_LINHAS | VERIFICA_COLUNAS | VERIFICA_BLOCOS)
5
6  int validSudokuSel(int s[DIM][DIM], int opcoes)
7  {
8      int i, j, k, valor;
9      int presente[DIM];
10     int temVazio = 0;
11     int bl, bc;
12
13     /* Domínio dos valores (sempre verificado). */
14     for (i = 1; i <= LADO; i++)
15         for (j = 1; j <= LADO; j++) {
16             valor = s[i][j];
17             if (valor < 0 || valor > LADO) return 0;
18             if (valor == VAZIO) temVazio = 1;
19         }

```

```

20
21     if (opcoes & VERIFICA_LINHAS)
22         for (i = 1; i <= LADO; i++) {
23             for (k = 0; k < DIM; k++) presente[k] = 0;
24             for (j = 1; j <= LADO; j++) {
25                 valor = s[i][j];
26                 if (valor != VAZIO) {
27                     if (presente[valor]) return 0;
28                     presente[valor] = 1;
29                 }
30             }
31         }
32
33     if (opcoes & VERIFICA_COLUNAS)
34         for (j = 1; j <= LADO; j++) {
35             for (k = 0; k < DIM; k++) presente[k] = 0;
36             for (i = 1; i <= LADO; i++) {
37                 valor = s[i][j];
38                 if (valor != VAZIO) {
39                     if (presente[valor]) return 0;
40                     presente[valor] = 1;
41                 }
42             }
43         }
44
45     if (opcoes & VERIFICA_BLOCOS)
46         for (bl = 0; bl < BLOCO; bl++)
47             for (bc = 0; bc < BLOCO; bc++) {
48                 for (k = 0; k < DIM; k++) presente[k] = 0;
49                 for (i = 1; i <= BLOCO; i++)
50                     for (j = 1; j <= BLOCO; j++) {
51                         valor = s[bl*BLOCO+i][bc*BLOCO+j];
52                         if (valor != VAZIO) {
53                             if (presente[valor]) return 0;
54                             presente[valor] = 1;
55                         }
56                     }
57             }
58     return temVazio ? 2 : 1;
59 }

```

Listing D.7: validSudoku com flags de verificação seletiva.

D.4.4 Programa gerador completo

```

1  int main(void)
2  {
3      int s[DIM][DIM] = {{0}};

```

```

4      long tentativas = 0;
5      const long LIMITE = 5000000L; /* limite prático */
6
7      srand((unsigned) time(NULL));
8
9      do {
10         permutations(s);
11         tentativas++;
12     } while (validSudokuSel(s, VERIFICA_COLUNAS | VERIFICA_BLOCOS) !=
13             1
14             && tentativas < LIMITE);
15
16     if (tentativas < LIMITE) {
17         printf("Sudoku gerado apos %ld tentativas:\n", tentativas);
18         imprimirTabuleiro(s);
19     } else {
20         printf("Limite de %ld tentativas atingido sem sucesso.\n",
21               LIMITE);
22         printf("(Isso e esperado: a probabilidade e da ordem de
23               10^-21.)\n");
24     }
25     return 0;
26 }

```

Listing D.8: Gerador por permutações aleatórias — prova de conceito.

D.4.5 Discussão do experimento

Executar este programa é instrutivo: na esmagadora maioria das execuções, o limite de tentativas é atingido sem sucesso. Esse comportamento, longe de ser uma falha do código, é uma *característica fundamental* do método: por isso é chamado de “força bruta”. O número aproximado de arranjos cuja primeira linha é qualquer permutação fixa e cujas demais linhas constituem um Sudoku válido é, segundo cálculos publicados na literatura, $6,67 \times 10^{21}$. Como existem $9!^9 \approx 1,1 \times 10^{50}$ combinações de nove permutações independentes, a probabilidade de “acerto ao acaso” é da ordem de 6×10^{-29} — isto é, virtualmente zero.

[gear] Observação de Engenharia de Software

Algoritmos puros de Monte Carlo, sem nenhuma poda, são úteis apenas como *baseline* pedagógico. O próximo passo natural é combinar o sorteio aleatório com algum mecanismo de poda — exatamente o que faremos quando unirmos esta ideia ao retrocesso, no gerador de quebra-cabeças (Seção D.8).

D.4.6 Variante prática — usando retrocesso para construir um Sudoku válido

Como exercício adicional, observe que podemos construir Sudokus válidos de modo *determinístico* embaralhando a ordem em que o solucionador do Desafio 2 testa os dígitos. Em vez de tentar sempre 1, 2, ..., 9, geramos uma permutação aleatória da sequência {1, 2, ..., 9} para cada célula vazia. O resultado é um Sudoku completo e *aleatório*, gerado em milissegundos. Esta é a

abordagem realmente útil na prática; a função `permutations` pura, com retorno via `validSudoku`, serve para demonstrar o conceito de força bruta.

D.5 Desafio 4: solucionador “fácil” — estratégia dos singletons

Enunciado do desafio

Implemente um solucionador para Sudokus “fáceis” baseado na repetição das estratégias humanas: para cada célula vazia, mantenha a lista de dígitos ainda possíveis (com base nos valores já fixados na linha, coluna e sub-grade); identifique singletons (célula com uma única possibilidade restante) e singletons ocultos (dígito que só cabe em uma célula da unidade) e os fixe; repita até que nenhuma estratégia produza progresso.

D.5.1 Análise do problema

As estratégias descritas no apêndice — singletons, duplas, duplas ocultas, triplas, triplas ocultas — podem ser implementadas em qualquer ordem, porque qualquer uma delas, ao agir, simplifica o tabuleiro para as demais. Vamos concentrar nossa atenção nas duas mais produtivas: o singleton “nu” (célula com uma única possibilidade) e o singleton “oculto” (dígito que aparece como possível em apenas uma célula de uma linha, coluna ou sub-grade). Essas duas estratégias resolvem, sozinhas, a grande maioria dos Sudokus de níveis fácil e médio.

Representação das possibilidades. Para cada célula vazia, queremos um conjunto de dígitos. A representação clássica em C é uma *máscara de bits*: usamos um `int` em que o bit $(d - 1)$ está ligado se, e somente se, o dígito d ainda é possível para aquela célula. Isso permite operações conjuntivas elegantes:

- `(mask & (1 << (d-1)))` testa se d pertence ao conjunto;
- `mask |= (1 << (d-1))` adiciona d ao conjunto;
- `mask &= ~(1 << (d-1))` remove d do conjunto;
- contar quantos bits estão ligados conta o tamanho do conjunto.

[gear] Observação de Engenharia de Software

Máscaras de bits são onipresentes em C quando precisamos manipular “conjuntos de até 32 (ou 64) elementos”. O custo de memória é mínimo, as operações são triviais e tudo cabe em registradores do processador. Em situações didáticas pode-se usar um array de 9 inteiros booleanos por célula, mas o ganho de clareza paga-se com perda de desempenho. Para Sudoku, a máscara é a escolha natural.

D.5.2 Algoritmo

1. **Calcule as possibilidades.** Para cada célula vazia, examine sua linha, sua coluna e sua sub-grade; o conjunto dos dígitos ausentes desses três conjuntos é a lista de possibilidades.
2. **Aplique singletons nus.** Se a lista de alguma célula contém um único dígito, fixe esse dígito na célula.

3. **Aplique singletons ocultos.** Para cada linha, cada coluna e cada sub-grade, e para cada dígito d de 1 a 9, conte em quantas células vazias dessa unidade o dígito d é possível. Se a contagem for 1, fixe d naquela célula.
4. **Repita.** Após qualquer alteração no tabuleiro, recalcule as possibilidades e reaplique as estratégias. Pare quando uma iteração completa não produzir nenhuma alteração.
5. **Veredito.** Se o tabuleiro está completo, sucesso. Caso contrário, este Sudoku exige técnicas mais avançadas (vide Seções D.5 e D.6).

D.5.3 Implementação

```
1  /* Retorna o número de bits ligados em mask. */
2  int contarBits(int mask)
3  {
4      int n = 0;
5      while (mask) {
6          n += mask & 1;
7          mask >>= 1;
8      }
9      return n;
10 }
11
12 /* Se mask tem exatamente um bit ligado, retorna o dígito      */
13 /* correspondente (1..9); caso contrário, retorna 0.          */
14 int unicoDigito(int mask)
15 {
16     int d;
17     if (contarBits(mask) != 1) return 0;
18     for (d = 1; d <= LADO; d++)
19         if (mask & (1 << (d - 1))) return d;
20     return 0;
21 }
```

Listing D.9: Funções auxiliares para manipulação de bitmasks.

```
1  /* Para cada célula vazia, registra em possib[i][j] uma máscara */
2  /* de bits dos dígitos ainda permitidos.                          */
3  void calcularPossibilidades(int s[DIM][DIM], int possib[DIM][DIM])
4  {
5      int i, j, d;
6      for (i = 1; i <= LADO; i++)
7          for (j = 1; j <= LADO; j++) {
8              if (s[i][j] != VAZIO) {
9                  possib[i][j] = 0;          /* já preenchida */
10                 continue;
11             }
12             possib[i][j] = 0;
```

```

13         for (d = 1; d <= LADO; d++)
14             if (ehMovimentoValido(s, i, j, d))
15                 possib[i][j] |= (1 << (d - 1));
16     }
17 }

```

Listing D.10: Cálculo da matriz de possibilidades para cada célula.

```

1  /* Percorre o tabuleiro fixando toda célula que tenha exatamente */
2  /* uma possibilidade. Retorna 1 se houve mudança, 0 se não.          */
3  int aplicarSingletonsNus(int s[DIM][DIM], int possib[DIM][DIM])
4  {
5      int i, j, d, mudou = 0;
6      for (i = 1; i <= LADO; i++)
7          for (j = 1; j <= LADO; j++)
8              if (s[i][j] == VAZIO) {
9                  d = unicoDigito(possib[i][j]);
10                 if (d != 0) {
11                     s[i][j] = d;
12                     possib[i][j] = 0;
13                     mudou = 1;
14                 }
15             }
16     return mudou;
17 }

```

Listing D.11: Aplicação dos singletons nus.

```

1  /* Para cada linha, coluna ou bloco, procura um dígito que só      */
2  /* pode ser colocado em uma única célula da unidade --- e o fixa.*/
3  int aplicarSingletonsOcultos(int s[DIM][DIM], int possib[DIM][DIM])
4  {
5      int i, j, d, mudou = 0;
6      int contagem, ii, jj, posI = 0, posJ = 0, br, bc;
7
8      /* Singletons ocultos em LINHAS. */
9      for (i = 1; i <= LADO; i++)
10         for (d = 1; d <= LADO; d++) {
11             contagem = 0;
12             for (j = 1; j <= LADO; j++)
13                 if (s[i][j] == VAZIO
14                     && (possib[i][j] & (1 << (d - 1)))) {
15                     contagem++;
16                     posJ = j;
17                 }
18             if (contagem == 1) {
19                 s[i][posJ] = d;
20                 possib[i][posJ] = 0;

```

```

21         mudou = 1;
22     }
23 }
24
25 /* Singletons ocultos em COLUNAS. */
26 for (j = 1; j <= LADO; j++)
27     for (d = 1; d <= LADO; d++) {
28         contagem = 0;
29         for (i = 1; i <= LADO; i++)
30             if (s[i][j] == VAZIO
31                 && (possib[i][j] & (1 << (d - 1)))) {
32                 contagem++;
33                 posI = i;
34             }
35         if (contagem == 1) {
36             s[posI][j] = d;
37             possib[posI][j] = 0;
38             mudou = 1;
39         }
40     }
41
42 /* Singletons ocultos em SUB-GRANDES 3x3. */
43 for (br = 0; br < BLOCO; br++)
44     for (bc = 0; bc < BLOCO; bc++)
45         for (d = 1; d <= LADO; d++) {
46             contagem = 0;
47             for (ii = 1; ii <= BLOCO; ii++)
48                 for (jj = 1; jj <= BLOCO; jj++) {
49                     i = br * BLOCO + ii;
50                     j = bc * BLOCO + jj;
51                     if (s[i][j] == VAZIO
52                         && (possib[i][j] & (1 << (d - 1)))) {
53                         contagem++;
54                         posI = i; posJ = j;
55                     }
56                 }
57             if (contagem == 1) {
58                 s[posI][posJ] = d;
59                 possib[posI][posJ] = 0;
60                 mudou = 1;
61             }
62         }
63 return mudou;
64 }

```

Listing D.12: Aplicação dos singletons ocultos.

```

1 /* Aplica as estratégias repetidamente, recalculando */

```

```

2  /* possibilidades após cada rodada de alterações. */
3  /* Retorna 1 se o tabuleiro foi totalmente resolvido. */
4  int resolverFacil(int s[DIM][DIM])
5  {
6      int possib[DIM][DIM];
7      int progresso, i, j, vazias;
8
9      do {
10         calcularPossibilidades(s, possib);
11         progresso = aplicarSingletonsNus(s, possib);
12
13         /* Recalcule antes da próxima estratégia para refletir */
14         /* as células recém-fixadas. */
15         calcularPossibilidades(s, possib);
16         progresso |= aplicarSingletonsOcultos(s, possib);
17     } while (progresso);
18
19     /* Conta células ainda vazias. */
20     vazias = 0;
21     for (i = 1; i <= LADO; i++)
22         for (j = 1; j <= LADO; j++)
23             if (s[i][j] == VAZIO) vazias++;
24
25     return vazias == 0;
26 }

```

Listing D.13: Solucionador completo para Sudokus fáceis.

D.5.4 Discussão e limitações

Esta versão resolve, sem qualquer suposição ou retrocesso, virtualmente todos os Sudokus rotulados como “fácil” por jornais e revistas. Para Sudokus médios, ela costuma ficar muito próxima da solução — frequentemente deixa apenas duas ou três células sem decisão. Para Sudokus difíceis, porém, ela trava muito antes do fim.

[check] Boa Prática de Programação

Recalcular as possibilidades *antes* de cada estratégia evita que informações estagnadas levem o algoritmo a deduções inválidas. O custo é pequeno (algumas dezenas de operações), e a robustez aumenta enormemente.

D.5.5 Comentário sobre duplas, triplas e suas versões ocultas

As estratégias de duplas, triplas, duplas ocultas e triplas ocultas descritas no Apêndice D são extensões naturais do raciocínio dos singletons: em vez de procurar “uma célula com uma possibilidade”, procuramos “duas células que dividem exatamente duas possibilidades”, ou “dois dígitos que só cabem em duas células de uma unidade”. A lógica de implementação é mais elaborada, mas sempre se reduz a percorrer cada unidade (linha, coluna ou sub-grade) e, dentro

dela, examinar combinações de tamanho 2 ou 3. Para os propósitos pedagógicos deste apêndice, os singletons (nus e ocultos) capturam a essência da técnica; deixamos duplas e triplas como extensão recomendada ao leitor.

D.6 Desafio 5: heurística “esperar antes de tomar uma decisão”

Enunciado do desafio

Implemente a heurística do tipo *esperar antes de tomar uma decisão*: (i) associe a cada célula vazia uma lista de possibilidades; (ii) caracterize o estado do tabuleiro contando o total de posicionamentos ainda possíveis; (iii) para cada movimento candidato, avalie a contagem do tabuleiro *após* esse movimento; (iv) escolha o movimento que deixar a contagem mais alta (em caso de empate, sorteie).

D.6.1 Análise do problema

A heurística é exatamente a oposta da intuição comum. Quando o problema é puramente combinatório — como no Passeio do Cavalo do Capítulo 6 e nas Oito Rainhas — a tentação é avançar rapidamente fechando opções. A filosofia “mantenha suas opções abertas” diz o contrário: *prefira a jogada que preserve o maior número de futuros possíveis*. A justificativa é simples: cada jogada que “fecha” o tabuleiro aumenta a probabilidade de que, mais adiante, alguma célula fique sem nenhum dígito viável — um beco sem saída.

A medida quantitativa proposta é a soma, sobre todas as células vazias do tabuleiro, do número de dígitos ainda permitidos. Essa contagem dá um “orçamento de liberdade” para o tabuleiro. Entre dois movimentos candidatos, preferimos o que deixar a contagem *maior*.

[gear] Observação de Engenharia de Software

Heurísticas, por natureza, não oferecem garantias. Esta, em particular, não é sempre superior ao retrocesso ingênuo — mas é especialmente eficaz em Sudokus de dificuldade intermediária, onde reduz dramaticamente o número de jogadas hipotéticas. É também uma excelente ferramenta de estudo: ela ilustra como uma decisão simples (qual jogada fazer agora?) pode ser informada por uma medida global do estado do problema.

D.6.2 Algoritmo

1. Defina `contarPossibilidades(s)` que soma, para cada célula vazia, o número de dígitos $d \in \{1, \dots, 9\}$ admissíveis (via `ehMovimentoValido`).
2. Para cada par (célula vazia, dígito admissível):
 - (a) Atribua hipoteticamente o dígito à célula.
 - (b) Compute a contagem global resultante.
 - (c) Restaure a célula a vazio.
3. Selecione o movimento de maior contagem. Em caso de empate, sorteie um dos empatados.
4. Efetue o movimento escolhido (definitivamente).

5. Repita até que o tabuleiro esteja completo ou até que algum movimento leve a contagem a zero (beco sem saída).
6. Em becos sem saída, retroceda ao último ponto de decisão — ou seja, combine a heurística com retrocesso.

D.6.3 Implementação

```

1  /* Soma, sobre cada célula vazia, o número de dígitos válidos. */
2  int contarPossibilidades(int s[DIM][DIM])
3  {
4      int i, j, d, total = 0;
5      for (i = 1; i <= LADO; i++)
6          for (j = 1; j <= LADO; j++)
7              if (s[i][j] == VAZIO)
8                  for (d = 1; d <= LADO; d++)
9                      if (ehMovimentoValido(s, i, j, d))
10                         total++;
11     return total;
12 }

```

Listing D.14: Contagem global de possibilidades.

```

1  /* Estrutura para representar um movimento candidato. */
2  typedef struct {
3      int i, j, d, contagem;
4  } Movimento;
5
6  /* Tenta cada movimento possível e elege o que preserva a maior */
7  /* contagem global de possibilidades. Retorna 1 se conseguiu */
8  /* efetuar um movimento; 0 se não há mais movimentos válidos. */
9  int escolherMovimentoEspera(int s[DIM][DIM])
10 {
11     Movimento empate[729]; /* no máximo 81 * 9 movimentos */
12     int nEmpate = 0;
13     int melhor = -1;
14     int i, j, d, c;
15
16     for (i = 1; i <= LADO; i++)
17         for (j = 1; j <= LADO; j++)
18             if (s[i][j] == VAZIO)
19                 for (d = 1; d <= LADO; d++)
20                     if (ehMovimentoValido(s, i, j, d)) {
21                         s[i][j] = d;
22                         c = contarPossibilidades(s);
23                         s[i][j] = VAZIO;
24
25                         if (c > melhor) {

```

```

26         melhor = c;
27         nEmpate = 0;
28         empate[nEmpate].i = i;
29         empate[nEmpate].j = j;
30         empate[nEmpate].d = d;
31         empate[nEmpate].contagem = c;
32         nEmpate++;
33     } else if (c == melhor) {
34         empate[nEmpate].i = i;
35         empate[nEmpate].j = j;
36         empate[nEmpate].d = d;
37         empate[nEmpate].contagem = c;
38         nEmpate++;
39     }
40 }
41
42 if (nEmpate == 0)
43     return 0;    /* nenhum movimento válido */
44
45 /* Em caso de empate, sorteia. */
46 int idx = rand() % nEmpate;
47 s[ empate[idx].i ][ empate[idx].j ] = empate[idx].d;
48 return 1;
49 }
50
51 int resolverHeuristica(int s[DIM][DIM])
52 {
53     int i, j, vazias;
54
55     do {
56         if (!escolherMovimentoEspera(s))
57             return 0;    /* beco sem saída */
58
59         vazias = 0;
60         for (i = 1; i <= LADO; i++)
61             for (j = 1; j <= LADO; j++)
62                 if (s[i][j] == VAZIO) vazias++;
63     } while (vazias > 0);
64
65     return 1;
66 }

```

Listing D.15: Heurística “esperar antes de tomar uma decisão”.

D.6.4 Discussão de complexidade

A cada movimento, examinamos no máximo $81 \times 9 = 729$ candidatos e, para cada um, recomputamos uma contagem que custa também 81×9 operações. O custo de uma única decisão é,

portanto, da ordem de $729 \times 729 \approx 5,3 \times 10^5$ operações. Como cada Sudoku exige até 81 decisões, o custo total fica em torno de $4,3 \times 10^7$ operações no pior caso — ainda perfeitamente factível em um computador moderno.

[bolt] Dica de Desempenho

O laço de contagem é o ponto quente do algoritmo. Em produção, vale a pena manter uma matriz de possibilidades (vide Desafio 4) e atualizá-la incrementalmente: quando colocamos d em `s[i][j]`, basta remover o bit correspondente das máscaras das células da mesma linha, coluna e bloco. Isso reduz o custo por movimento a $O(27)$ em vez de $O(729)$.

[warning] Erro Comum de Programação

Esquecer de restaurar a célula a `VAZIO` após a avaliação hipotética é um bug particularmente difícil de diagnosticar: o tabuleiro vai acumulando movimentos fantasmas, e o algoritmo passa a “ver” contagens sistematicamente mais baixas, levando a escolhas erradas. Use sempre o padrão “**tente, meça, desfaça**”.

D.6.5 Combinando heurística e retrocesso

A heurística pura, sem retrocesso, é capaz de resolver Sudokus difíceis em muitas execuções, mas eventualmente atinge becos sem saída. A forma robusta é manter uma pilha de pontos de decisão: cada vez que escolhemos entre vários candidatos empatados, salvamos o estado do tabuleiro. Se mais adiante chegamos a um beco sem saída, restauramos o estado salvo mais recente e tentamos outro empatado. Em essência, transformamos a heurística em um guia para o retrocesso: ela diz *qual ramo* explorar primeiro, enquanto o retrocesso garante a correteude.

D.7 Desafio 6: heurística da antecipação (look-ahead)

Enunciado do desafio

A heurística da antecipação é um aprimoramento da heurística de “esperar antes de tomar uma decisão”. No caso de empate, antecipe mais uma colocação: para cada movimento empatado, avalie qual seria a contagem do tabuleiro após *duas* jogadas e prefira aquele cujo melhor desdobramento for o mais alto.

D.7.1 Análise do problema

A heurística da Seção D.6 às vezes não consegue distinguir entre vários movimentos com o mesmo número de possibilidades imediatas. Quando isso ocorre, fazer um sorteio é honesto, mas ingênuo — estamos ignorando informação que está apenas “um passo mais à frente”. A heurística da antecipação propõe: para cada candidato empatado, simule a próxima jogada (a melhor possível, segundo o mesmo critério) e use a contagem *após dois movimentos* como critério de desempate.

[gear] Observação de Engenharia de Software

Estamos aplicando, em pequena escala, um princípio universal de inteligência artificial: *busca em profundidade limitada*. O xadrez, o Go e dezenas de outros jogos usam a mesma ideia, geralmente em profundidades muito maiores e com poda (alfa-beta) para evitar explorar ramos óbvios. A diferença

é que, no Sudoku, a função de avaliação é simples e exata (a contagem global), enquanto em jogos adversariais ela é apenas uma estimativa.

D.7.2 Algoritmo

1. Compute todos os movimentos candidatos como antes, registrando sua contagem imediata.
2. Identifique a melhor contagem imediata.
3. Filtre apenas os movimentos empatados nesse máximo.
4. Para cada empatado, simule a jogada e, no tabuleiro resultante, compute a melhor contagem imediata da próxima jogada (sem efetivamente jogá-la — apenas avalie). Esse valor é a *contagem antecipada* do candidato.
5. Restaure a célula simulada.
6. Selecione o candidato com a maior contagem antecipada. Em caso de novo empate, sorteie.

D.7.3 Implementação

```

1  /* Dado um tabuleiro, encontra a melhor contagem imediata          */
2  /* alcançável por algum movimento (sem efetuar-lo).                */
3  int melhorContagemImediata(int s[DIM][DIM])
4  {
5      int i, j, d, c, melhor = -1;
6      for (i = 1; i <= LADO; i++)
7          for (j = 1; j <= LADO; j++)
8              if (s[i][j] == VAZIO)
9                  for (d = 1; d <= LADO; d++)
10                     if (ehMovimentoValido(s, i, j, d)) {
11                         s[i][j] = d;
12                         c = contarPossibilidades(s);
13                         s[i][j] = VAZIO;
14                         if (c > melhor) melhor = c;
15                     }
16      return melhor;
17  }
18
19  /* Substitui escolherMovimentoEspera com critério de antecipação. */
20  int escolherMovimentoAntecipacao(int s[DIM][DIM])
21  {
22      Movimento empate[729];
23      int nEmpate = 0;
24      int melhorImediata = -1;
25      int i, j, d, c;
26
27      /* Fase 1: coletar empatados na melhor contagem imediata.      */
28      for (i = 1; i <= LADO; i++)

```

```

29     for (j = 1; j <= LADO; j++)
30         if (s[i][j] == VAZIO)
31             for (d = 1; d <= LADO; d++)
32                 if (ehMovimentoValido(s, i, j, d)) {
33                     s[i][j] = d;
34                     c = contarPossibilidades(s);
35                     s[i][j] = VAZIO;
36
37                     if (c > melhorImediata) {
38                         melhorImediata = c;
39                         nEmpate = 0;
40                     }
41                     if (c == melhorImediata) {
42                         empate[nEmpate].i = i;
43                         empate[nEmpate].j = j;
44                         empate[nEmpate].d = d;
45                         empate[nEmpate].contagem = c;
46                         nEmpate++;
47                     }
48                 }
49
50     if (nEmpate == 0)
51         return 0;
52
53     /* Fase 2: se houver um único empatado, jogue-o. */
54     if (nEmpate == 1) {
55         s[empate[0].i][empate[0].j] = empate[0].d;
56         return 1;
57     }
58
59     /* Fase 3: desempate por antecipação. */
60     Movimento finalistas[729];
61     int nFinalistas = 0;
62     int melhorAntecipada = -1;
63
64     for (int k = 0; k < nEmpate; k++) {
65         s[empate[k].i][empate[k].j] = empate[k].d;
66         int antecipada = melhorContagemImediata(s);
67         s[empate[k].i][empate[k].j] = VAZIO;
68
69         if (antecipada > melhorAntecipada) {
70             melhorAntecipada = antecipada;
71             nFinalistas = 0;
72         }
73         if (antecipada == melhorAntecipada) {
74             finalistas[nFinalistas++] = empate[k];
75         }
76     }

```

```

77
78     int idx = rand() % nFinalistas;
79     s[finalistas[idx].i][finalistas[idx].j] = finalistas[idx].d;
80     return 1;
81 }

```

Listing D.16: Heurística com antecipação de uma jogada.

D.7.4 Custo e benefício

A antecipação custa caro: para cada empatado, examinamos novamente todos os movimentos candidatos. No pior caso, isso eleva o custo por decisão a $O(n^4)$, com $n = 9$. Na prática, porém, o número de empatados costuma ser pequeno (de 2 a 5 na maioria dos tabuleiros), de modo que o tempo extra é tolerável. O benefício é uma redução notável no número de becos sem saída: tabuleiros que exigiriam retrocesso resolvem-se, muitas vezes, sem qualquer back-up.

[check] Boa Prática de Programação

Heurísticas com look-ahead profundo são, em geral, sujeitas à lei dos rendimentos decrescentes: olhar dois movimentos à frente compensa quase sempre; olhar três compensa às vezes; olhar quatro raramente paga seu custo. Comece simples; aprofunde apenas se medições indicarem benefício.

D.8 Desafio 7: criando quebra-cabeças com células vazias

Enunciado do desafio

Com um Sudoku completo em mãos, esvazie algumas células para criar um quebra-cabeça. Considere duas estratégias: (i) escolha aleatória de células a esvaziar; (ii) esvaziamento simétrico, usando a célula “reflexiva” — definida como $s[10-i][10-j]$ para a célula $s[i][j]$ — de modo a deixar o tabuleiro com simetria visual.

D.8.1 Análise do problema

A geração de Sudokus completos é o que fizemos no Desafio 2 (com retrocesso) ou no Desafio 3 (com permutações aleatórias). A criação de um *quebra-cabeça* é o passo seguinte: removemos algumas células, deixando para o jogador o trabalho de redescobri-las. O número de células removidas é um *proxy* da dificuldade: tabuleiros mais “vazios” tendem a ser mais difíceis, embora a relação não seja estrita (há Sudokus com poucos espaços que são surpreendentemente difíceis e vice-versa).

Simetria reflexiva. Sudokus publicados em jornais quase sempre apresentam um padrão simétrico de células fixas, por estética. A célula “reflexiva” de $s[i][j]$ é $s[10-i][10-j]$ — esta é a involução $i \mapsto 10-i$, $j \mapsto 10-j$, equivalente a uma rotação de 180° em torno do centro do tabuleiro. Note dois casos importantes:

- Se $i = 5$ e $j = 5$, a célula é seu próprio reflexo (centro).

- Se (i, j) e $(10 - i, 10 - j)$ são distintas, esvaziar uma obriga a esvaziar a outra para preservar a simetria.

D.8.2 Algoritmo (esvaziamento aleatório simples)

1. Receba um Sudoku completo e uma quantidade-alvo `numCelulasParaEsvaziar`.
2. Enquanto o número de células esvaziadas for menor que o alvo:
 - (a) Sorteie aleatoriamente $i, j \in \{1, \dots, 9\}$.
 - (b) Se `s[i][j]` ainda estiver preenchida, esvazie-a.

D.8.3 Algoritmo (esvaziamento simétrico)

1. Sorteie $i, j \in \{1, \dots, 9\}$.
2. Calcule $i' = 10 - i$ e $j' = 10 - j$.
3. Se ambas `s[i][j]` e `s[i'][j']` estão preenchidas: esvazie ambas. Trate o caso especial $i = i'$ e $j = j'$ (centro) como um único esvaziamento.

D.8.4 Implementação

```

1  /* Esvazia aleatoriamente n células de um Sudoku completo.          */
2  void esvaziarAleatorio(int s[DIM][DIM], int n)
3  {
4      int esvaziadas = 0;
5      int i, j;
6      while (esvaziadas < n) {
7          i = (rand() % LADO) + 1;
8          j = (rand() % LADO) + 1;
9          if (s[i][j] != VAZIO) {
10             s[i][j] = VAZIO;
11             esvaziadas++;
12         }
13     }
14 }
15
16 /* Esvazia n células de modo a preservar a simetria reflexiva.      */
17 /* n é o número-alvo de células esvaziadas; o algoritmo pode       */
18 /* parar com até n+1 esvaziadas (quando o último par é central). */
19 void esvaziarSimetrico(int s[DIM][DIM], int n)
20 {
21     int esvaziadas = 0;
22     int i, j, i2, j2;
23     while (esvaziadas < n) {
24         i = (rand() % LADO) + 1;
25         j = (rand() % LADO) + 1;
26         i2 = (LADO + 1) - i;    /* 10 - i */

```

```

27     j2 = (LADO + 1) - j;    /* 10 - j */
28
29     if (s[i][j] != VAZIO && s[i2][j2] != VAZIO) {
30         if (i == i2 && j == j2) {
31             /* Célula central: conta como um único esvaziamento.
32                */
33             s[i][j] = VAZIO;
34             esvaziadas++;
35         } else {
36             s[i][j] = VAZIO;
37             s[i2][j2] = VAZIO;
38             esvaziadas += 2;
39         }
40     }
41 }

```

Listing D.17: Dois esquemas de esvaziamento de células.

[warning] Erro Comum de Programação

A inicialização do gerador de números aleatórios com `srand` é frequentemente esquecida. Sem ela, `rand()` produz a mesma sequência em cada execução — e o programa gera sempre o mesmo “Sudoku aleatório”. Use `srand( (unsigned) time(NULL) )` logo no início de `main`.

D.8.5 Sobre dificuldade e solvibilidade

Um detalhe sutil: o esvaziamento desimpedido pode gerar tabuleiros com *mais de uma* solução. Por exemplo, se removermos todas as células de uma sub-grade, ela ficará vazia, e quaisquer duas atribuições permutáveis das nove células gerarão Sudokus distintos. Para gerar quebra-cabeças “apropriados”, o ideal é, após cada esvaziamento, verificar se o tabuleiro ainda tem solução única (vide Seção D.9); se não, restaurar a última remoção. Esse aprimoramento transforma o gerador em uma ferramenta verdadeiramente útil.

D.9 Desafio 8: demonstrando que um Sudoku tem solução única**Enunciado do desafio**

Os Sudokus publicados em geral têm exatamente uma solução. Desenvolva um meio de demonstrar que um jogo específico tem exatamente uma solução.

D.9.1 Análise do problema

A primeira tentação é tentar gerar *todas* as soluções e contar. Para Sudokus mal definidos com poucas pistas, isso pode levar tempo proibitivo — existem Sudokus mal definidos com mais de 10^{15} soluções. Felizmente, a pergunta que precisamos responder é mais modesta: “há exatamente uma solução?”. Isso significa que basta descobrir se há *pelo menos duas*; ao encontrar a segunda, podemos parar imediatamente.

A estratégia é adaptar o solucionador por retrocesso do Desafio 2: em vez de retornar logo após encontrar uma solução, ele continua a busca e conta. Como só nos interessa a distinção “uma solução / mais de uma”, abortamos a busca assim que a contagem atinge 2. Essa poda é o que torna o algoritmo prático.

[gear] Observação de Engenharia de Software

Este é um exemplo clássico de *poda baseada no objetivo*: não precisamos da resposta exata para uma pergunta caríssima; bastaria distinguir entre “zero”, “um” e “dois ou mais”. Reformular a pergunta para que esse limiar seja pequeno transforma um problema intratável em um problema rotineiro.

D.9.2 Algoritmo

1. Receba o tabuleiro a verificar.
2. Copie-o (a verificação não deve alterar o original).
3. Execute uma busca por retrocesso na cópia, contando soluções.
4. Aborte a busca assim que o contador atinja 2.
5. Retorne “única” se o contador for 1; “múltiplas” se for 2; “sem solução” se for 0.

D.9.3 Implementação

```

1  /* Conta soluções até o limite indicado. Modifica s (caller deve
   * copiar). */
2  int contarSolucoes(int s[DIM][DIM], int limite)
3  {
4      int i, j, d, total = 0;
5      for (i = 1; i <= LADO; i++) {
6          for (j = 1; j <= LADO; j++) {
7              if (s[i][j] == VAZIO) {
8                  for (d = 1; d <= LADO; d++)
9                      if (ehMovimentoValido(s, i, j, d)) {
10                         s[i][j] = d;
11                         total += contarSolucoes(s, limite - total);
12                         s[i][j] = VAZIO;
13                         if (total >= limite)
14                             return total;      /* poda */
15                     }
16                 return total;
17             }
18         }
19     }
20     /* Sem células vazias: encontramos uma solução completa. */
21     return 1;
22 }
23
24 /* Verifica se o tabuleiro tem exatamente uma solução. */

```

```

25  /* Retorna: */
26  /* 0 -> sem solução */
27  /* 1 -> exatamente uma solução */
28  /* 2 -> duas ou mais soluções */
29  int contarUnicidade(int original[DIM][DIM])
30  {
31      int copia[DIM][DIM];
32      int i, j;
33
34      /* Copia o tabuleiro para preservar o original. */
35      for (i = 0; i < DIM; i++)
36          for (j = 0; j < DIM; j++)
37              copia[i][j] = original[i][j];
38
39      return contarSolucoes(copia, 2);
40  }

```

Listing D.18: Contagem limitada de soluções e teste de unicidade.

D.9.4 Discussão da poda

A chave dessa solução está na linha `if (total >= limite) return total`. Note que “`limite = 2`” significa: “conte até no máximo duas soluções; assim que achar a segunda, pare”. A passagem `limite - total` na chamada recursiva propaga o “orçamento de soluções” restante. Isso transforma o algoritmo, que no pior caso é exponencial, em algo cujo custo escala apenas com a árvore necessária para encontrar até duas soluções.

[check] Boa Prática de Programação

Sempre copiar a entrada antes de executar uma operação destrutiva permite que o chamador continue a usar o tabuleiro original. A regra geral é: “funções de consulta não devem modificar seus argumentos”. Quando a modificação é inevitável (por eficiência), documente explicitamente esse contrato.

D.9.5 Aplicação prática: refinando o gerador de quebra-cabeças

Vamos integrar este teste de unicidade ao gerador do Desafio 7 para produzir Sudokus que tenham, por construção, solução única.

```

1  /* Gera um quebra-cabeça com simetria reflexiva e solução única. */
2  /* Retorna o número efetivo de células esvaziadas (pode ser menor */
3  /* que o solicitado se o algoritmo esgotar tentativas). */
4  int gerarSudokuValido(int s[DIM][DIM],
5                        int completo[DIM][DIM],
6                        int numAlvo)
7  {
8      int i, j;
9      int esvaziadas = 0;

```

```

10     int tentativas = 0;
11     const int MAX_TENT = 200;
12
13     /* Começa com o tabuleiro completo. */
14     for (i = 0; i < DIM; i++)
15         for (j = 0; j < DIM; j++)
16             s[i][j] = completo[i][j];
17
18     while (esvaziadas < numAlvo && tentativas < MAX_TENT) {
19         tentativas++;
20         int ii = (rand() % LADO) + 1;
21         int jj = (rand() % LADO) + 1;
22         int i2 = (LADO + 1) - ii;
23         int j2 = (LADO + 1) - jj;
24
25         if (s[ii][jj] == VAZIO || s[i2][j2] == VAZIO)
26             continue;
27
28         /* Salva os valores antes de esvaziar. */
29         int salvoA = s[ii][jj];
30         int salvoB = s[i2][j2];
31
32         s[ii][jj] = VAZIO;
33         s[i2][j2] = VAZIO;
34
35         /* Verifica unicidade. */
36         if (contarUnicidade(s) == 1) {
37             /* Pode manter o esvaziamento. */
38             esvaziadas += (ii == i2 && jj == j2) ? 1 : 2;
39             tentativas = 0;
40         } else {
41             /* Restaura para preservar unicidade. */
42             s[ii][jj] = salvoA;
43             s[i2][j2] = salvoB;
44         }
45     }
46     return esvaziadas;
47 }

```

Listing D.19: Gerador de Sudokus com solução única garantida.

[bolt] Dica de Desempenho

Este gerador é, deliberadamente, “conservador”: prefere manter um quebra-cabeça com solução única do que se arriscar a um Sudoku ambíguo. O contador `tentativas` é zerado a cada esvaziamento bem-sucedido e atinge `MAX_TENT` apenas quando o algoritmo conclui que remoções adicionais provocariam ambiguidade. O resultado típico contém de 50 a 60 células preenchidas, faixa que abrange Sudokus de dificuldade média a difícil.

D.10 Programa integrado e demonstração

A Listagem D.20 reúne os componentes principais em um único `main` que demonstra o ciclo completo: geramos um Sudoku completo via retrocesso randomizado, esvaziamos células simetricamente preservando solução única e, ao final, resolvemos o quebra-cabeça gerado.

```
1  /* Versão de resolverBacktracking que tenta dígitos em ordem          */
2  /* aleatória, produzindo Sudokus completos aleatórios.              */
3  /*                                                                    */
4  int gerarCompletoAleatorio(int s[DIM][DIM])
5  {
6      int i, j, k, d, tmp;
7      int ordem[DIM];
8
9      for (i = 1; i <= LADO; i++)
10         for (j = 1; j <= LADO; j++)
11             if (s[i][j] == VAZIO) {
12
13                 /* Constrói e embaralha a ordem dos dígitos 1..9. */
14                 for (k = 1; k <= LADO; k++) ordem[k] = k;
15                 for (k = LADO; k > 1; k--) {
16                     int r = (rand() % k) + 1;
17                     tmp = ordem[k]; ordem[k] = ordem[r]; ordem[r] =
18                         tmp;
19                 }
20
21                 for (k = 1; k <= LADO; k++) {
22                     d = ordem[k];
23                     if (ehMovimentoValido(s, i, j, d)) {
24                         s[i][j] = d;
25                         if (gerarCompletoAleatorio(s)) return 1;
26                         s[i][j] = VAZIO;
27                     }
28                 }
29                 return 0;
30             }
31
32 return 1;
33 }
34
35 int main(void)
36 {
37     int completo[DIM][DIM] = {{0}};
38     int puzzle[DIM][DIM];
39     int copia[DIM][DIM];
40     int i, j, n;
41
42     srand((unsigned) time(NULL));
```

```

41  /* 1) Gera um Sudoku completo aleatório. */
42  if (!gerarCompletoAleatorio(completo)) {
43      printf("Falha ao gerar Sudoku completo.\n");
44      return 1;
45  }
46  printf("Sudoku completo gerado:");
47  imprimirTabuleiro(completo);
48
49  /* 2) Esvazia 50 células com simetria, preservando unicidade. */
50  n = gerarSudokuValido(puzzle, completo, 50);
51  printf("\nQuebra-cabeça com %d células esvaziadas:", n);
52  imprimirTabuleiro(puzzle);
53
54  /* 3) Resolve o quebra-cabeça para confirmar. */
55  for (i = 0; i < DIM; i++)
56      for (j = 0; j < DIM; j++) copia[i][j] = puzzle[i][j];
57
58  if (resolverBacktracking(copia)) {
59      printf("\nSolução reconstruída:");
60      imprimirTabuleiro(copia);
61  } else {
62      printf("\nERRO: o quebra-cabeça não tem solução.\n");
63  }
64  return 0;
65 }

```

Listing D.20: Programa integrado.

[check] Boa Prática de Programação

Manter cópias separadas para “solução de referência” e “cópia de trabalho” é a única forma robusta de verificar se um quebra-cabeça é consistente. Tentar economizar memória reaproveitando o mesmo array para os dois propósitos costuma se traduzir em horas de depuração.

D.11 Conclusão

Neste apêndice, partimos da definição de um quebra-cabeça aparentemente simples — preencher uma grade de 9×9 com os dígitos de 1 a 9 obedecendo a três restrições — e construímos, passo a passo, soluções que vão da força bruta cega à programação de heurísticas sofisticadas e à geração automática de novos quebra-cabeças. Mais que um exercício de jogo, o Sudoku serviu como um campo de provas para diversos paradigmas que serão utilizados ao longo de todo o livro:

- **Encapsulamento via funções utilitárias.** Funções como `ehMovimentoValido` e `validSudoku` foram escritas *uma única vez* e reaproveitadas em todos os solucionadores.
- **Recursão como ferramenta natural.** O retrocesso ficou elegantemente expresso por uma função recursiva curta. Voltaremos a esse padrão ao tratarmos do Passeio do Cavalo, das Oito Rainhas, da Torre de Hanói e da travessia de árvores.

- **Estruturas de iteração aninhadas.** O domínio bidimensional do Sudoku exigiu o uso disciplinado dos laços `for`, `while` e `do...while` aprendidos no Capítulo 4.
- **Geração de números pseudoaleatórios.** A função `rand` permite explorar técnicas *Monte Carlo* e introduzir variação nos geradores — mas requer cuidado com a inicialização da semente.
- **Manipulação eficiente de conjuntos.** As máscaras de bits do Desafio 4 são um padrão idiomático em C, especialmente útil quando o universo é pequeno e fixo.
- **Heurísticas.** A heurística da Seção D.6 e a antecipação da Seção D.7 oferecem um primeiro contato com técnicas que dominam a inteligência artificial moderna.
- **Poda baseada no objetivo.** O Desafio 8 mostra como reformular uma pergunta cara em uma pergunta barata, decisão arquitetural que faz toda a diferença em problemas combinatórios.

[gear] Observação de Engenharia de Software

A história da computação contém numerosos exemplos em que problemas considerados “insolúveis” tornaram-se rotineiros graças não apenas ao avanço do hardware, mas à invenção de algoritmos mais inteligentes. O Sudoku é uma metáfora em miniatura desse processo: a força bruta cega (9^{81} possibilidades) jamais terminaria, ao passo que o retrocesso bem implementado resolve qualquer Sudoku publicado em milissegundos.

Os recursos de programação aqui exercitados constituem, todos eles, ferramentas com as quais o leitor já se tornou íntimo nos Capítulos 1 a 7. Cabe ao leitor agora estender este material: implementar duplas, duplas ocultas, triplas e triplas ocultas; medir empiricamente quais combinações de estratégias produzem os solucionadores mais eficientes; gerar quebra-cabeças classificados por dificuldade; e — por que não? — adicionar uma interface gráfica usando a biblioteca *Allegro* apresentada no Apêndice E, que escapa do padrão da linguagem C mas recompensa o trabalho com programas verdadeiramente bonitos.

Bom Sudoku!